

```

import plotly.graph_objects as go
import plotly.express as px
from sklearn.metrics.pairwise import cosine_similarity
from google.colab import files
import io
import warnings
warnings.filterwarnings('ignore')

# Set style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

print("✅ All packages imported successfully!")
print("=" * 70)

# PART 1: DATA INPUT OPTIONS

print("\n📊 GROCERY BASKET SIMILARITY ANALYZER")
print("=" * 70)
print("\nChoose your data input method:")
print("1. Upload CSV file")
print("2. Use sample data")
print("3. Create custom baskets manually")
print("=" * 70)

data_choice = input("\nEnter your choice (1/2/3): ").strip()

# Option 1: Upload CSV File

if data_choice == "1":
    print("\n📁 Please upload your CSV file...")
    print("Expected format:")
    print("  - First column: Customer/Basket Name")
    print("  - Other columns: Item names with quantities")
    print("  Example:")
    print("  Customer,milk,bread,eggs,butter,cheese")
    print("  Customer A,2,1,1,1,1")
    print("  Customer B,1,2,1,0,0")
    print()

```

```

uploaded = files.upload()

for filename in uploaded.keys():
    df = pd.read_csv(io.BytesIO(uploaded[filename]))
    print(f"\n✅ File '{filename}' loaded successfully!")
    print(f"    Shape: {df.shape[0]} baskets × {df.shape[1]-1}
items")
    break

# Set first column as index (customer names)
df.set_index(df.columns[0], inplace=True)
df = df.fillna(0).astype(int)

# Option 2: Use Sample Data
elif data_choice == "2":
    print(f"\n✅ Loading sample data...")

    sample_data = {
        'Customer A': {'milk': 2, 'bread': 1, 'eggs': 1, 'butter': 1,
'cheese': 1, 'yogurt': 0, 'jam': 0},
        'Customer B': {'milk': 1, 'bread': 2, 'eggs': 1, 'butter': 0,
'cheese': 0, 'yogurt': 2, 'jam': 0},
        'Customer C': {'milk': 2, 'bread': 0, 'eggs': 2, 'butter': 1,
'cheese': 2, 'yogurt': 1, 'jam': 0},
        'Customer D': {'milk': 0, 'bread': 3, 'eggs': 0, 'butter': 1,
'cheese': 0, 'yogurt': 0, 'jam': 2},
        'Customer E': {'milk': 1, 'bread': 0, 'eggs': 0, 'butter': 1,
'cheese': 1, 'yogurt': 3, 'jam': 0},
        'Customer F': {'milk': 3, 'bread': 2, 'eggs': 2, 'butter': 2,
'cheese': 1, 'yogurt': 1, 'jam': 0},
        'Customer G': {'milk': 0, 'bread': 1, 'eggs': 0, 'butter': 0,
'cheese': 0, 'yogurt': 0, 'jam': 3},
    }

    df = pd.DataFrame(sample_data).T
    print(f"    Shape: {df.shape[0]} baskets × {df.shape[1]} items")

    df = pd.DataFrame(sample_data).T
    print(f"    Shape: {df.shape[0]} baskets × {df.shape[1]} items")

```

```

# Option 3: Create Custom Baskets
else:
    print("\n🛒 Create your custom baskets!")

    # Get number of baskets
    num_baskets = int(input("\nEnter number of baskets: "))

    # Get items
    items_input = input("Enter item names (comma-separated, e.g., milk,bread,eggs): ")
    items = [item.strip() for item in items_input.split(',')]

    # Create empty dataframe
    data = {}

    print("\n" + "=" * 70)
    print("Enter quantities for each basket (press Enter for 0)")
    print("=" * 70)

    for i in range(num_baskets):
        basket_name = input(f"\nBasket {i+1} name (or press Enter for 'Basket {i+1}'): ").strip()
        if not basket_name:
            basket_name = f"Basket {i+1}"

        basket_items = {}
        print(f"\nEnter quantities for {basket_name}:")

        for item in items:
            qty = input(f"    {item}: ").strip()
            basket_items[item] = int(qty) if qty else 0

        data[basket_name] = basket_items

    df = pd.DataFrame(data).T
    print(f"\n✅ Created {df.shape[0]} baskets with {df.shape[1]} items")

# PART 2: DATA EXPLORATION
#

print("\n" + "=" * 70)
print("📊 DATA OVERVIEW")

```

```

print("=" * 70)

print("\n1. Basket-Item Matrix:")
display(df)

print("\n2. Summary Statistics:")
display(df.describe())

print("\n3. Total items per basket:")
display(df.sum(axis=1).sort_values(ascending=False))

print("\n4. Item popularity (total quantity across all baskets):")
display(df.sum(axis=0).sort_values(ascending=False))

# PART 3: VISUALIZATIONS

print("\n" + "=" * 70)
print("📊 GENERATING VISUALIZATIONS")
print("=" * 70)

# Visualization 1: Heatmap of baskets
fig, ax = plt.subplots(figsize=(12, 8))
sns.heatmap(df, annot=True, fmt='d', cmap='YlOrRd', linewidths=0.5,
            cbar_kws={'label': 'Quantity'}, ax=ax)
plt.title('Grocery Basket Heatmap\n(Quantities of Items per Customer)',
          fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Items', fontsize=12, fontweight='bold')
plt.ylabel('Customers', fontsize=12, fontweight='bold')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

print("✅ Heatmap generated!")

# Visualization 2: Bar chart of item popularity
fig, ax = plt.subplots(figsize=(10, 6))
item_totals = df.sum(axis=0).sort_values(ascending=True)
colors = plt.cm.viridis(np.linspace(0, 1, len(item_totals)))
item_totals.plot(kind='barh', color=colors, ax=ax)
plt.title('Item Popularity Across All Baskets', fontsize=16,
          fontweight='bold', pad=20)
plt.xlabel('Total Quantity', fontsize=12, fontweight='bold')

```

```

plt.ylabel('Items', fontsize=12, fontweight='bold')
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

print("✅ Bar chart generated!")

# Visualization 3: Basket size comparison
fig, ax = plt.subplots(figsize=(10, 6))
basket_totals = df.sum(axis=1).sort_values(ascending=True)
colors = plt.cm.plasma(np.linspace(0, 1, len(basket_totals)))
basket_totals.plot(kind='barh', color=colors, ax=ax)
plt.title('Total Items per Basket', fontsize=16, fontweight='bold',
pad=20)
plt.xlabel('Total Quantity', fontsize=12, fontweight='bold')
plt.ylabel('Customers', fontsize=12, fontweight='bold')
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

print("✅ Basket comparison generated!")

# PART 4: COSINE SIMILARITY CALCULATION

print("\n" + "=" * 70)
print("📊 CALCULATING COSINE SIMILARITY")
print("=" * 70)

# Calculate cosine similarity matrix
similarity_matrix = cosine_similarity(df)
similarity_df = pd.DataFrame(similarity_matrix,
                             index=df.index,
                             columns=df.index)

print("\n✅ Similarity Matrix:")
display(similarity_df.round(3))

# Visualize similarity matrix
fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(similarity_df, annot=True, fmt='.3f', cmap='coolwarm',
            center=0.5, linewidths=0.5, square=True,
            cbar_kws={'label': 'Cosine Similarity'}, ax=ax)

```

```

plt.title('Cosine Similarity Matrix\n(1.0 = Identical, 0.0 = No
Similarity)',
         fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Customers', fontsize=12, fontweight='bold')
plt.ylabel('Customers', fontsize=12, fontweight='bold')
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

print("✅ Similarity matrix visualization generated!")

# PART 5: SIMILARITY ANALYSIS FOR SELECTED BASKET

print("\n" + "=" * 70)
print("🎯 BASKET SIMILARITY ANALYSIS")
print("=" * 70)

print("\nAvailable baskets:")
for idx, basket in enumerate(df.index, 1):
    print(f"{idx}. {basket}")

basket_choice = int(input("\nSelect a basket number to analyze: ")) - 1
selected_basket = df.index[basket_choice]

print(f"\n📊 Analyzing similarities for: {selected_basket}")
print("=" * 70)

# Get similarities for selected basket
similarities =
similarity_df[selected_basket].sort_values(ascending=False)[1:]

print("\n🏆 Top Similar Baskets:")
print("-" * 70)
for rank, (basket, score) in enumerate(similarities.head(5).items(),
1):
    similarity_percent = score * 100

    if score >= 0.7:
        label = "🟢 HIGH"
    elif score >= 0.4:
        label = "🟡 MEDIUM"

```

```

else:
    label = "🔴 LOW"

    print(f"{rank}. {basket:20s} | {label:12s} | {similarity_percent:6.2f}%")

    # Show vector comparison
    vec1 = df.loc[selected_basket].values
    vec2 = df.loc[basket].values
    print(f"    {selected_basket}: {vec1}")
    print(f"    {basket}: {vec2}")
    print()

# Visualization: Top similarities
fig, ax = plt.subplots(figsize=(10, 6))
top_similarities = similarities.head(5).sort_values()
colors = ['#d62728' if x < 0.4 else '#ff7f0e' if x < 0.7 else '#2ca02c'
          for x in top_similarities.values]
top_similarities.plot(kind='barh', color=colors, ax=ax)
plt.title(f'Top 5 Similar Baskets to "{selected_basket}"',
          fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Cosine Similarity Score', fontsize=12, fontweight='bold')
plt.ylabel('Customers', fontsize=12, fontweight='bold')
plt.xlim(0, 1)
plt.grid(axis='x', alpha=0.3)
plt.axvline(x=0.7, color='green', linestyle='--', alpha=0.5,
            label='High (≥0.7)')
plt.axvline(x=0.4, color='orange', linestyle='--', alpha=0.5,
            label='Medium (≥0.4)')
plt.legend()
plt.tight_layout()
plt.show()

print("✅ Similarity chart generated!")

# PART 6: SMART PRODUCT RECOMMENDATIONS

print("\n" + "=" * 70)
print("💡 SMART PRODUCT RECOMMENDATIONS")
print("=" * 70)

# Get items from selected basket

```

```

selected_items = set(df.columns[df.loc[selected_basket] > 0])

# Analyze top 3 similar baskets for recommendations
top_similar = similarities.head(3)
item_scores = {}

for basket, similarity_score in top_similar.items():
    basket_items = df.loc[basket]
    for item in df.columns:
        if item not in selected_items and basket_items[item] > 0:
            item_scores[item] = item_scores.get(item, 0) +
similarity_score

# Sort and get top recommendations
recommendations = sorted(item_scores.items(), key=lambda x: x[1],
reverse=True)

print(f"\n🛒 Recommended items for {selected_basket}:")
print("-" * 70)

if recommendations:
    for rank, (item, score) in enumerate(recommendations[:5], 1):
        confidence = (score / top_similar.sum()) * 100
        print(f"{rank}. {item:15s} | Confidence: {confidence:5.1f}% |
Score: {score:.3f}")
else:
    print("No recommendations available (customer has all popular
items)")

# Visualization: Recommendations
if recommendations:
    fig, ax = plt.subplots(figsize=(10, 6))
    rec_df = pd.DataFrame(recommendations[:5], columns=['Item',
'Score'])
    colors_rec = plt.cm.viridis(np.linspace(0, 1, len(rec_df)))
    ax.barh(rec_df['Item'], rec_df['Score'], color=colors_rec)
    plt.title(f'Top Recommended Items for "{selected_basket}"',
        fontsize=16, fontweight='bold', pad=20)
    plt.xlabel('Recommendation Score', fontsize=12, fontweight='bold')
    plt.ylabel('Items', fontsize=12, fontweight='bold')
    plt.grid(axis='x', alpha=0.3)
    plt.tight_layout()
    plt.show()

```



```

print("\n✅ Recommendations chart generated!")

#
=====
=====
# PART 7: INTERACTIVE VISUALIZATION (PLOTLY)
#
=====
=====

print("\n" + "=" * 70)
print("🎨 GENERATING INTERACTIVE VISUALIZATIONS")
print("=" * 70)

# Interactive 3D scatter plot using PCA for dimensionality reduction
from sklearn.decomposition import PCA

if df.shape[0] > 2:
    pca = PCA(n_components=min(3, df.shape[0], df.shape[1]))
    coords = pca.fit_transform(df)

    # Add selected basket highlighting
    colors_3d = ['red' if idx == selected_basket else 'blue' for idx in
df.index]
    sizes = [15 if idx == selected_basket else 10 for idx in df.index]

    if coords.shape[1] >= 3:
        fig = go.Figure(data=[go.Scatter3d(
            x=coords[:, 0],
            y=coords[:, 1],
            z=coords[:, 2],
            mode='markers+text',
            marker=dict(size=sizes, color=colors_3d, opacity=0.8),
            text=df.index,
            textposition="top center",
            hovertemplate='<b>{text}</b><br>X: {x:.2f}<br>Y:
{y:.2f}<br>Z: {z:.2f}<extra></extra>'
        )])

        fig.update_layout(
            title='3D Basket Similarity Visualization (PCA)',

```

```

        scene=dict(xaxis_title='PC1', yaxis_title='PC2',
zaxis_title='PC3'),
        height=700
    )
    fig.show()
    print("✅ Interactive 3D plot generated!")

# Interactive similarity network
fig = go.Figure()

# Add similarity threshold slider
threshold = 0.3

for i, basket1 in enumerate(df.index):
    for j, basket2 in enumerate(df.index):
        if i < j and similarity_df.loc[basket1, basket2] > threshold:
            fig.add_trace(go.Scatter(
                x=[i, j],
                y=[similarity_df.loc[basket1, basket2],
similarity_df.loc[basket1, basket2]],
                mode='lines',
                line=dict(color='lightgray',
width=similarity_df.loc[basket1, basket2]*3),
                showlegend=False,
                hoverinfo='skip'
            ))

fig.add_trace(go.Scatter(
    x=list(range(len(df.index))),
    y=[1] * len(df.index),
    mode='markers+text',
    marker=dict(size=20, color='royalblue'),
    text=df.index,
    textposition="top center",
    showlegend=False
))

fig.update_layout(
    title='Basket Similarity Network (Threshold > 0.3)',
    xaxis=dict(showgrid=False, zeroline=False, showticklabels=False),
    yaxis=dict(showgrid=False, zeroline=False, showticklabels=False,
range=[0.5, 1.5]),
    height=500

```

```

)
fig.show()

print("✅ Interactive network plot generated!")

#
=====
===
# PART 8: SUMMARY REPORT
#
=====
=====

print("\n" + "=" * 70)
print("📋 ANALYSIS SUMMARY REPORT")
print("=" * 70)

print(f"""
🎯 OVERALL STATISTICS
-----
• Total Baskets Analyzed: {len(df)}
• Total Unique Items: {len(df.columns)}
• Average Basket Size: {df.sum(axis=1).mean():.2f} items
• Most Popular Item: {df.sum(axis=0).idxmax()} ({df.sum(axis=0).max()}
total)
• Least Popular Item: {df.sum(axis=0).idxmin()} ({df.sum(axis=0).min()}
total)

🎯 SIMILARITY INSIGHTS FOR "{selected_basket}"
-----
• Most Similar Basket: {similarities.idxmax()} (Score:
{similarities.max():.3f})
• Least Similar Basket: {similarities.idxmin()} (Score:
{similarities.min():.3f})
• Average Similarity: {similarities.mean():.3f}
• Items in Basket: {int(df.loc[selected_basket].sum())}

🎯 ALGORITHM DETAILS
-----
• Method: Cosine Similarity
• Formula:  $\cos(\theta) = (A \cdot B) / (||A|| \times ||B||)$ 
• Range: 0.0 (no similarity) to 1.0 (identical)
• Vector Dimensions: {len(df.columns)}

```

🎯 RECOMMENDATIONS

```
• Total Recommendations Generated: {len(recommendations)}
• Top Recommended Item: {recommendations[0][0] if recommendations else
'None'}
• Recommendation Confidence:
{f' {(recommendations[0][1]/top_similar.sum()*100):.1f}%' if
recommendations else 'N/A'}
"""
```

```
print("=" * 70)
```

```
print("✅ ANALYSIS COMPLETE!")
```

```
print("=" * 70)
```

```
#
```

=====

=====

```
# BONUS: EXPORT RESULTS
```

```
#
```

=====

=====

```
export_choice = input("\n💾 Do you want to export results? (y/n):
").strip().lower()
```

```
if export_choice == 'y':
```

```
    # Export similarity matrix
```

```
    similarity_df.to_csv('similarity_matrix.csv')
```

```
    print("✅ Similarity matrix exported to 'similarity_matrix.csv'")
```

```
    # Export recommendations
```

```
    if recommendations:
```

```
        rec_df = pd.DataFrame(recommendations, columns=['Item',
'Score'])
```

```
        rec_df.to_csv(f'recommendations_{selected_basket.replace(" ",
"_" )}.csv', index=False)
```

```
        print(f"✅ Recommendations exported to
'recommendations_{selected_basket.replace(' ', '_')}.csv'")
```

```
    # Download files
```

```
    files.download('similarity_matrix.csv')
```

```
    if recommendations:
```

```
files.download(f'recommendations_{selected_basket.replace(" ",
"_")}.csv')

print("\n🎉 Thank you for using Grocery Basket Similarity Analyzer!")
print("=" * 70)
```